BrainyAI

# RAPPORT DE PROJET

Réalisé par :
Hiba EL IDRYSY
Ayoub AOUBID
Abderrazak MOULOUA

Date de soumission :
06/12/2005

# Résumé Exécutif

Ce projet a pour objectif de développer une plateforme web interactive basée sur le framework **Django**, permettant de tester et d'évaluer un ensemble d'algorithmes d'apprentissage automatique (Machine Learning).

Notre plateforme **BrainyAI** permet aux utilisateurs d'effectuer des prédictions à partir de modèles entraînés (Régression linéaire et logistique, SVM, Arbres de décision, Random Forest, XGBoost) appliqués à différents jeux de données (datasets). Elle offre également une fonctionnalité de **nettoyage de données**, permettant aux utilisateurs de traiter leurs propres jeux de données via une page dédiée, basée sur un script Python spécifique `(cleaning.py)`.

Les performances des modèles ont été évaluées à travers deux cas d'usage principaux :

- **La prédiction du prix d'un trajet en taxi (problème de régression).**
- **Le diagnostic du diabète (problème de classification).**

# Introduction

### Contexte et motivation :

L'intelligence artificielle occupe aujourd'hui une place centrale dans de nombreux domaines tels que la santé, le transport, la finance ou encore l'éducation. Cependant, la mise en place et l'exploitation des modèles d'apprentissage automatique nécessitent souvent des compétences techniques avancées en programmation, en mathématiques et en science des données.

La motivation principale de ce projet est donc de démocratiser l'accès aux algorithmes de Machine Learning en proposant une plateforme web conviviale, intuitive et accessible, permettant aux utilisateurs de tester et d'exploiter ces modèles sans avoir besoin d'une expertise approfondie.

### Objectifs du projet :

1. Comprendre et implémenter les principaux algorithmes d'apprentissage automatique (régression et classification).
2. Maîtriser les étapes essentielles du prétraitement des données, notamment le nettoyage des données (**data cleaning**).
3. Intégrer ces modèles dans une application web interactive et fonctionnelle basée sur le framework **Django**.

### Problématiques abordées :

- Comment traiter efficacement des données brutes contenant des valeurs manquantes, des incohérences ou des erreurs de format ?
- Quel algorithme offre les meilleures performances selon la nature du problème à résoudre (régression ou classification) ?

# Présentation des Datasets :

## A. Dataset de classification : Pima Indians Diabetes

Ce jeu de données est largement utilisé comme référence dans le domaine du Machine Learning appliqué à la santé. Son objectif principal est de prédire si une patiente présente ou non des signes de diabète, à partir de plusieurs paramètres médicaux.

- **Source :** https://github.com/jbrownlee/Datasets/blob/master/pima-indians-diabetes.csv
- **Volumétrie :** 768 lignes et 9 colonnes
- **Variable cible (Target) :** Prediction
  - **1 :** test positif au diabète
  - **0 :** test négatif

---

**Prétraitement effectué :**

- Remplacement des valeurs aberrantes par la borne supérieure lorsqu'elles dépassent la limite haute, et par la borne inférieure lorsqu'elles sont en dessous de la limite basse (technique de capping).
- Standardisation des données afin de mettre toutes les variables sur la même échelle et d'améliorer les performances des modèles.

## B. Dataset de régression : Taxi Trip Pricing

Ce jeu de données concerne la tarification des courses de taxi. L'objectif principal est d'estimer le prix total d'une course de taxi à partir de plusieurs caractéristiques telles que la distance, la durée, le type de paiement ou encore le nombre de passagers.

- **Source :** https://www.kaggle.com/datasets/denkuznetz/taxi-price-prediction
- **Volumétrie :** 1000 lignes et 11 colonnes
- **Variable cible (Target) :** Trip_Price

---

**Prétraitement effectué :**

- Remplacement des valeurs manquantes par le mode pour les colonnes catégorielles.
- Encodage des variables catégorielles à l'aide de la bibliothèque LabelEncoder.
- Remplacement des valeurs aberrantes par la borne supérieure lorsqu'elles dépassent la limite haute, et par la borne inférieure lorsqu'elles sont en dessous de la limite basse (technique de capping).

**Note :** les étapes de prétraitement seront détaillées plus en profondeur dans le paragraphe dédié à la page de nettoyage des données, où le fonctionnement du script cleaning.py et les différentes techniques utilisées seront expliqués de manière approfondie.

# Algorithme implémenté :

## 1. La régression linéaire

La régression linéaire est l'un des modèles les plus simples et les plus utilisés en Machine Learning pour prédire une valeur numérique continue. Elle consiste à trouver la droite qui s'ajuste le mieux aux données, en minimisant l'erreur quadratique moyenne (Mean Squared Error), soit à l'aide de la descente de gradient, soit via les équations normales.

Ce modèle est utilisé lorsque la relation entre la variable cible et les variables explicatives est approximativement linéaire. Après la préparation des données, le modèle est entraîné, puis évalué à l'aide d'indicateurs tels que le $R^2$ ou la MSE.

- **Avantage :** Simple, rapide et interprétable, la régression linéaire représente une excellente base de référence pour des tâches comme la tarification ou la prédiction de valeurs numériques.
- **Limites :** Elle reste toutefois sensible aux valeurs aberrantes et peu adaptée aux relations non linéaires complexes.

## 2. La régression logistique

La régression logistique est un modèle statistique largement utilisé pour les problèmes de classification binaire. Elle vise à séparer deux classes en estimant, pour chaque observation, une probabilité d'appartenance à une classe, comprise entre 0 et 1. Cette probabilité est obtenue grâce à la fonction sigmoïde, appliquée à une combinaison linéaire des caractéristiques d'entrée.

Le processus d'apprentissage consiste à ajuster les coefficients du modèle par descente de gradient, en minimisant une fonction de coût, généralement l'entropie croisée (log loss). Avant l'entraînement, les données doivent être préparées, notamment par la normalisation ou l'encodage des variables catégorielles. La décision finale est prise en comparant la probabilité estimée à un seuil, généralement fixé à 0,5.

- **Avantage :** Simple, rapide et interprétable, la régression logistique est particulièrement adaptée à des tâches telles que le diagnostic du diabète.
- **Limites :** Elle reste cependant limitée lorsqu'il existe des relations non linéaires complexes ou en présence de nombreuses valeurs aberrantes.

## 3. L'arbre de décision

Un arbre de décision est un modèle prédictif qui segmente les données en posant une succession de questions basées sur les caractéristiques d'entrée. À chaque nœud, l'algorithme choisit la variable et le seuil qui permettent de rendre les sous-groupes les plus homogènes possible, en se basant sur des critères tels que l'indice de Gini ou l'entropie (pour la classification), ou la variance (pour la régression).

- **Avantage :** Classification et Régression. Règles "Si... Alors...".
- **Limites :** Fort risque de surapprentissage (overfitting).

# Suite des Algorithmes :

## 4. Random Forest

La Random Forest (forêt aléatoire) est une méthode d'ensemble qui combine plusieurs arbres de décision afin d'améliorer les performances et la robustesse du modèle. Chaque arbre est entraîné sur un sous-ensemble différent du jeu de données, obtenu par la technique du bootstrap, et utilise un sous-ensemble aléatoire de variables à chaque division.

En classification, la prédiction finale est déterminée par un vote majoritaire de l'ensemble des arbres, tandis qu'en régression, elle correspond à la moyenne des prédictions individuelles. Cette approche permet de réduire la variance du modèle et de limiter le surapprentissage.

- **Avantage :** Robuste, performante et capable de mesurer l'importance des variables, la Random Forest est très utilisée pour des problèmes complexes, notamment pour la prédiction du diabète ou des prix de courses de taxi.
- **Limites :** En contrepartie, elle est plus coûteuse en calcul et moins interprétable qu'un arbre de décision unique.

## 5. XGBoost

XGBoost (Extreme Gradient Boosting) est une méthode de boosting très performante qui construit les arbres de manière séquentielle. Contrairement à la Random Forest, où les arbres sont indépendants, XGBoost ajoute à chaque étape un nouvel arbre qui cherche à corriger les erreurs commises par les modèles précédents, à l'aide d'une optimisation basée sur la descente de gradient.

Cette approche est combinée à des techniques de régularisation qui permettent de réduire le surapprentissage et d'améliorer la généralisation du modèle.

- **Avantage :** Très utilisé en compétition et dans les applications industrielles, XGBoost offre d'excellentes performances en classification comme en régression, notamment pour des données complexes ou bruitées, Il constitue un choix privilégié lorsque la précision est un critère majeur, comme pour la prédiction du diabète ou du prix d'un trajet en taxi.
- **Limites :** Cependant, il nécessite un réglage précis des hyperparamètres et une puissance de calcul plus importante.

## 6. SVM (Machines à vecteurs de support)

Les Machines à Vecteurs de Support (SVM) sont des modèles puissants utilisés pour la classification et la régression (SVR). Leur objectif est de trouver l'hyperplan qui sépare au mieux les classes, tout en maximisant la marge, c'est-à-dire la distance entre cet hyperplan et les points les plus proches, appelés vecteurs de support.

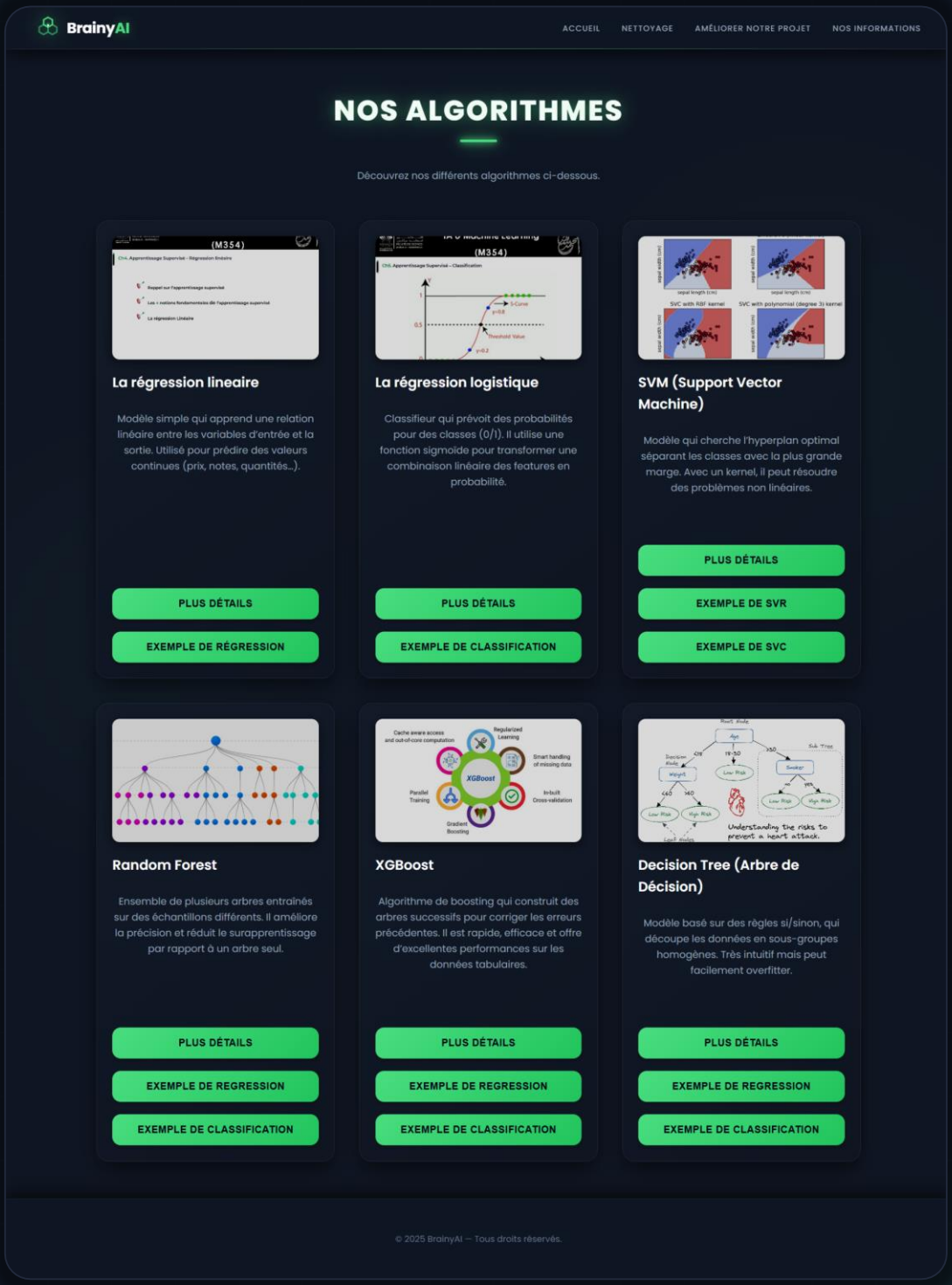
Les SVM peuvent également utiliser des fonctions noyau (kernels) afin de traiter des problèmes non linéaires en projetant les données dans un espace de dimension supérieure. Cette capacité les rend efficaces pour des jeux de données complexes ou bruités.

Malgré leur performance, les SVM peuvent devenir coûteuses en calcul pour de très grands jeux de données et nécessitent un réglage minutieux de leurs hyperparamètres.

# Résultat :

## Captures d’écran de l’application :

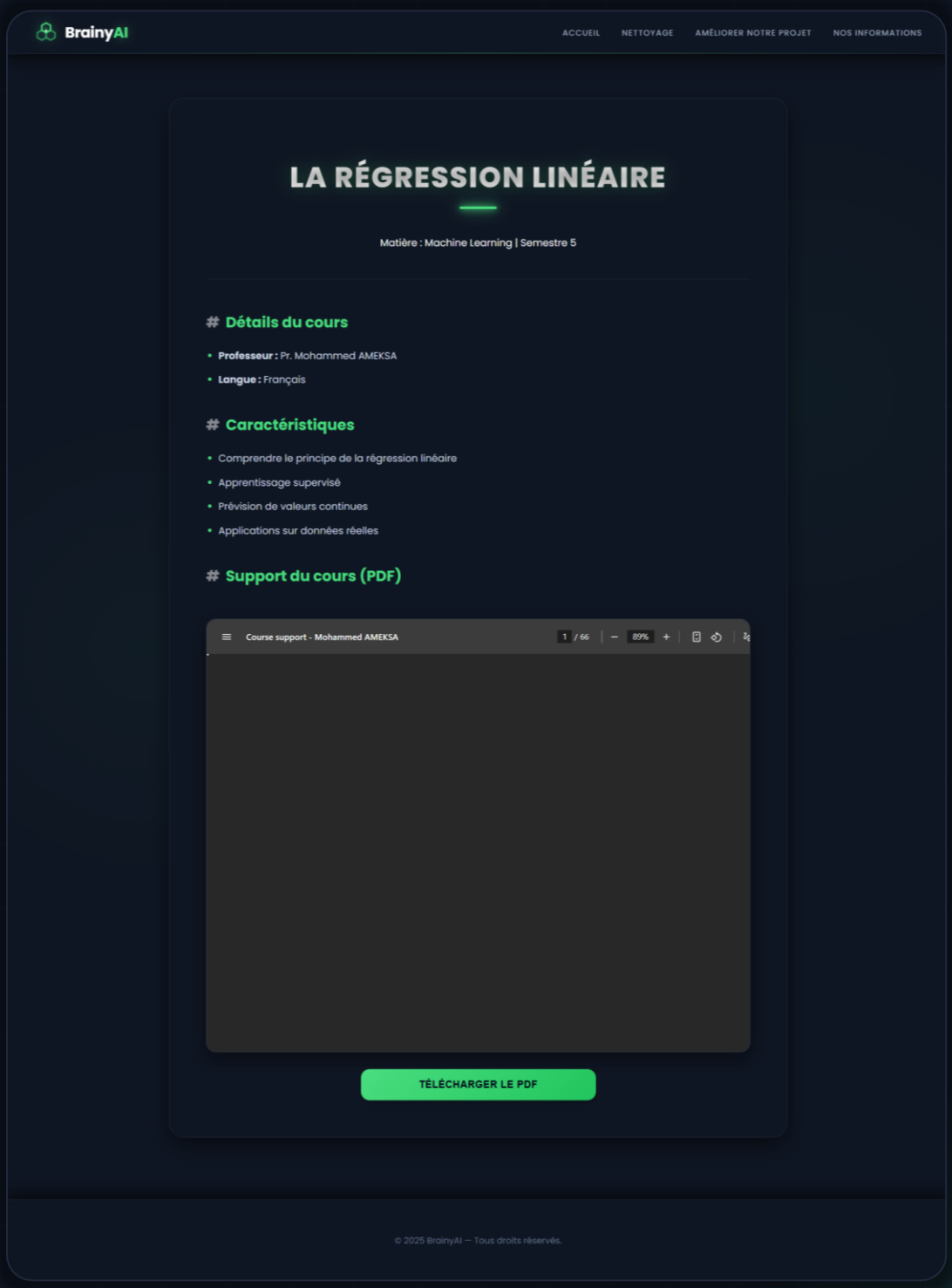
### Page d’accueil:



# Résultat :

## Captures d’écran de l’application :

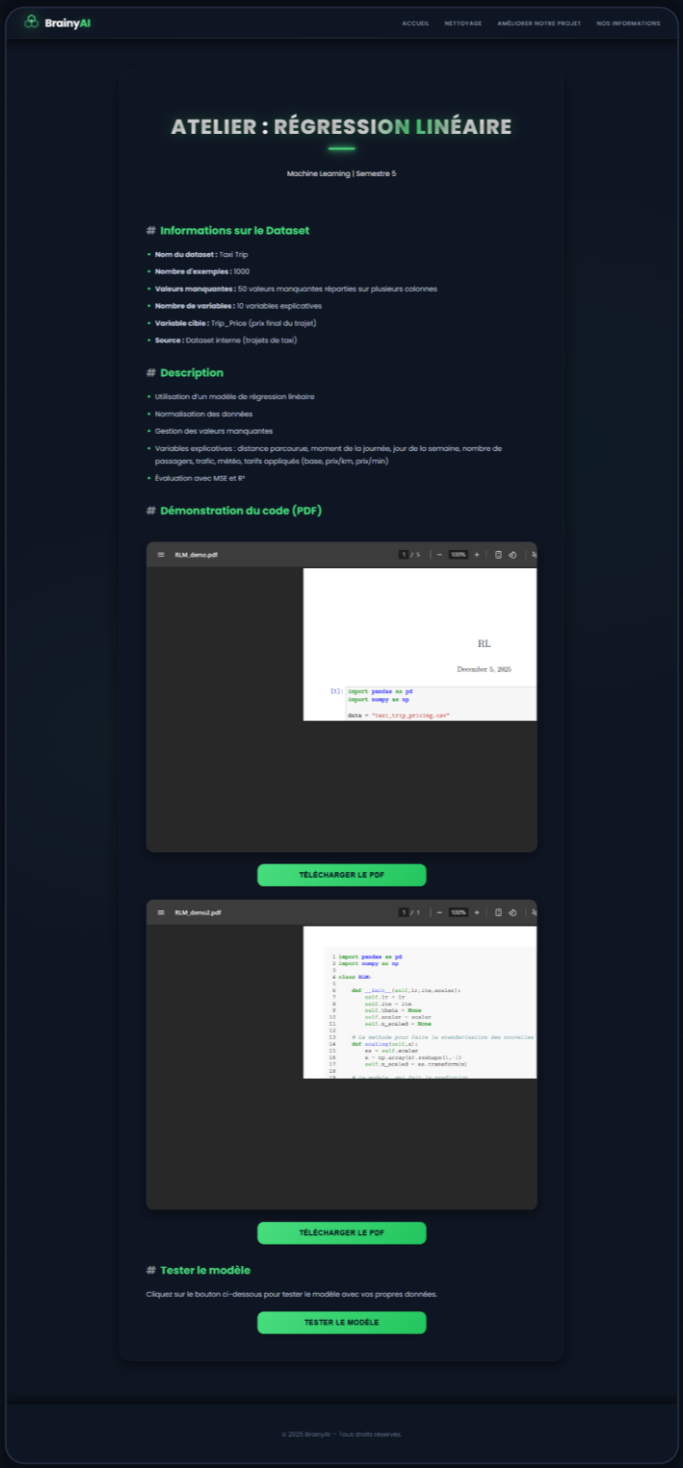
### Page de détails:



# Résultat :

## Captures d’écran de l’application :

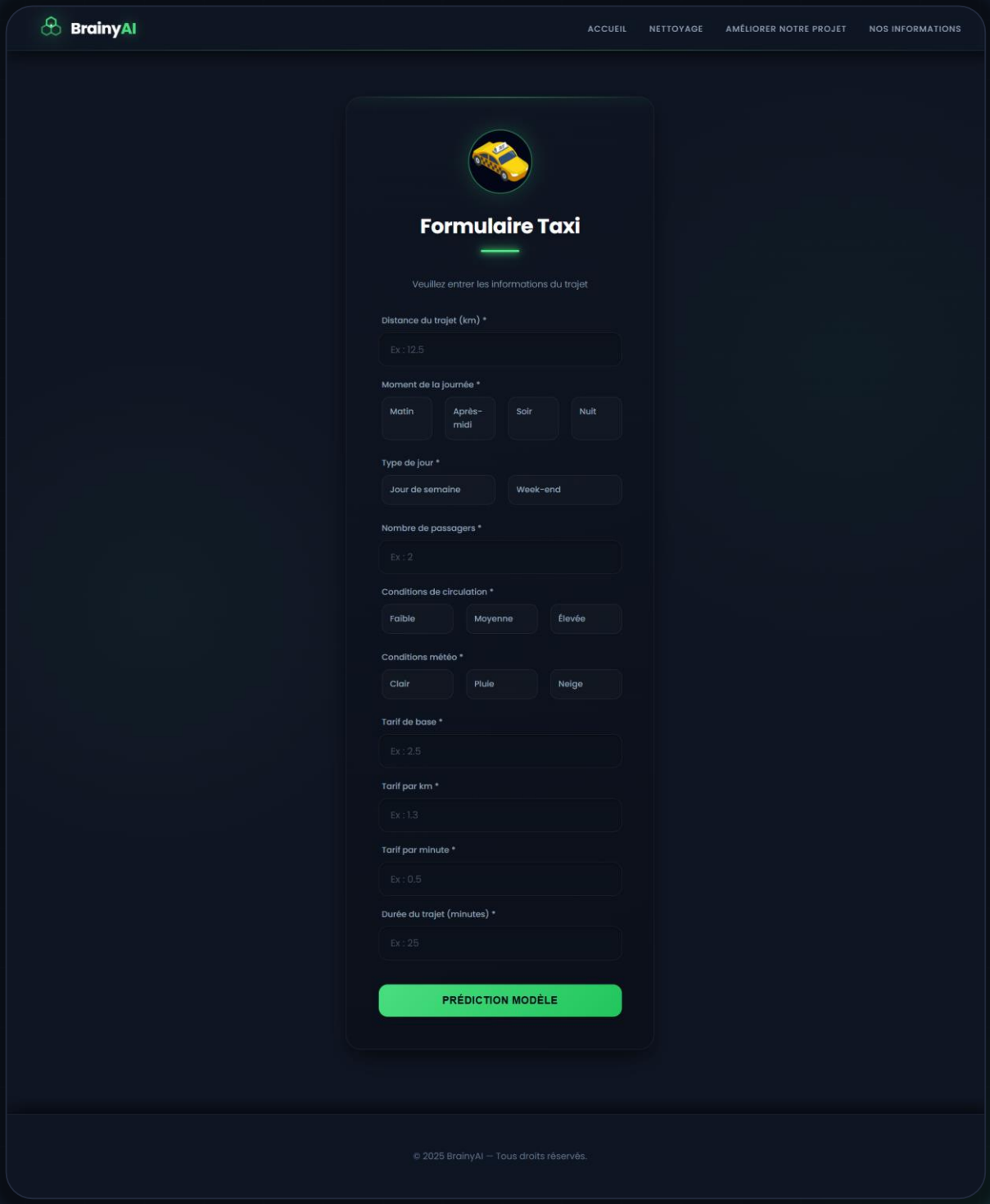
### Page d’atelier:



# Résultat :

## Captures d’écran de l’application :

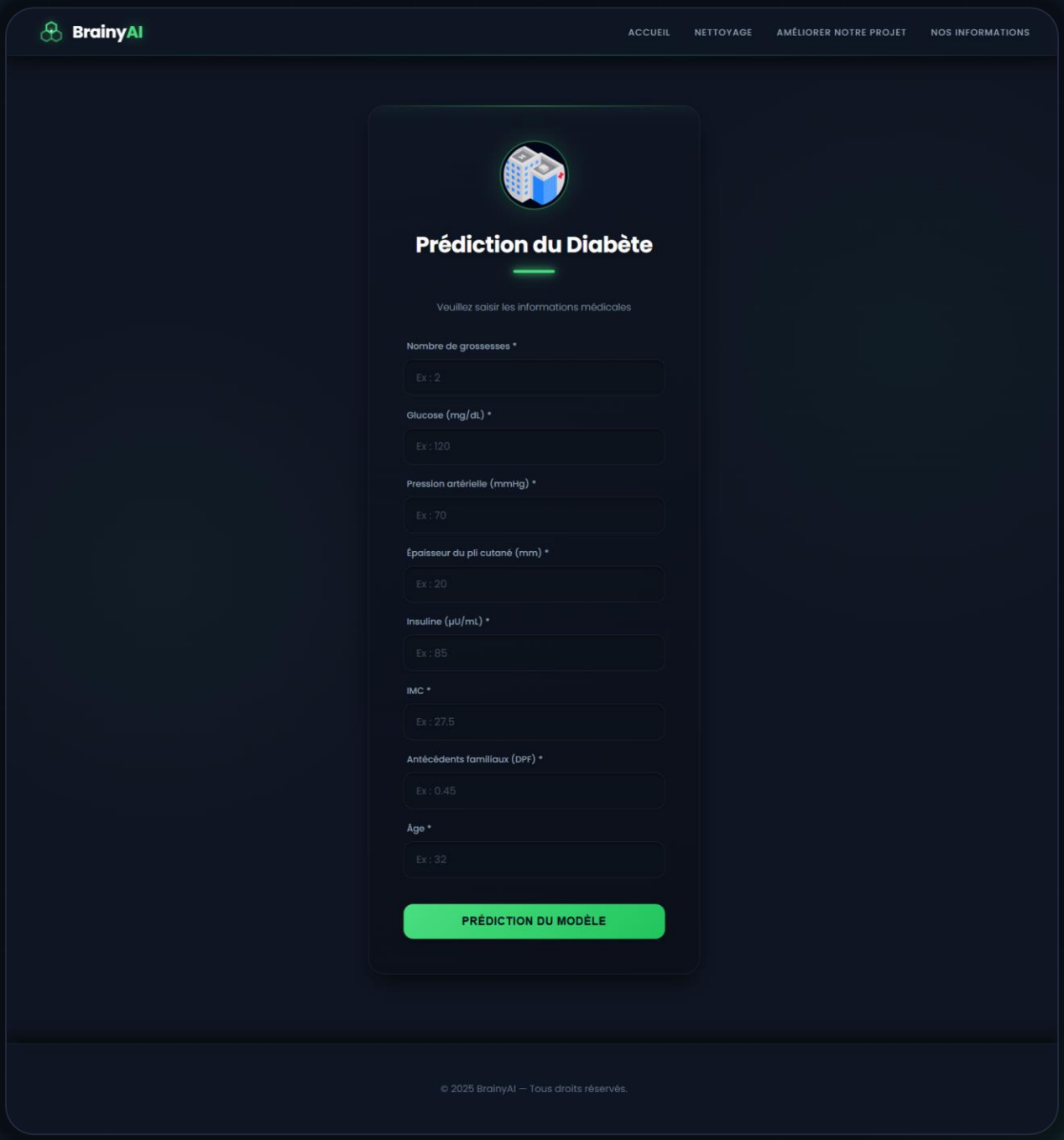
### Page de formulaire (Régression):



# Résultat :

## Captures d’écran de l’application :

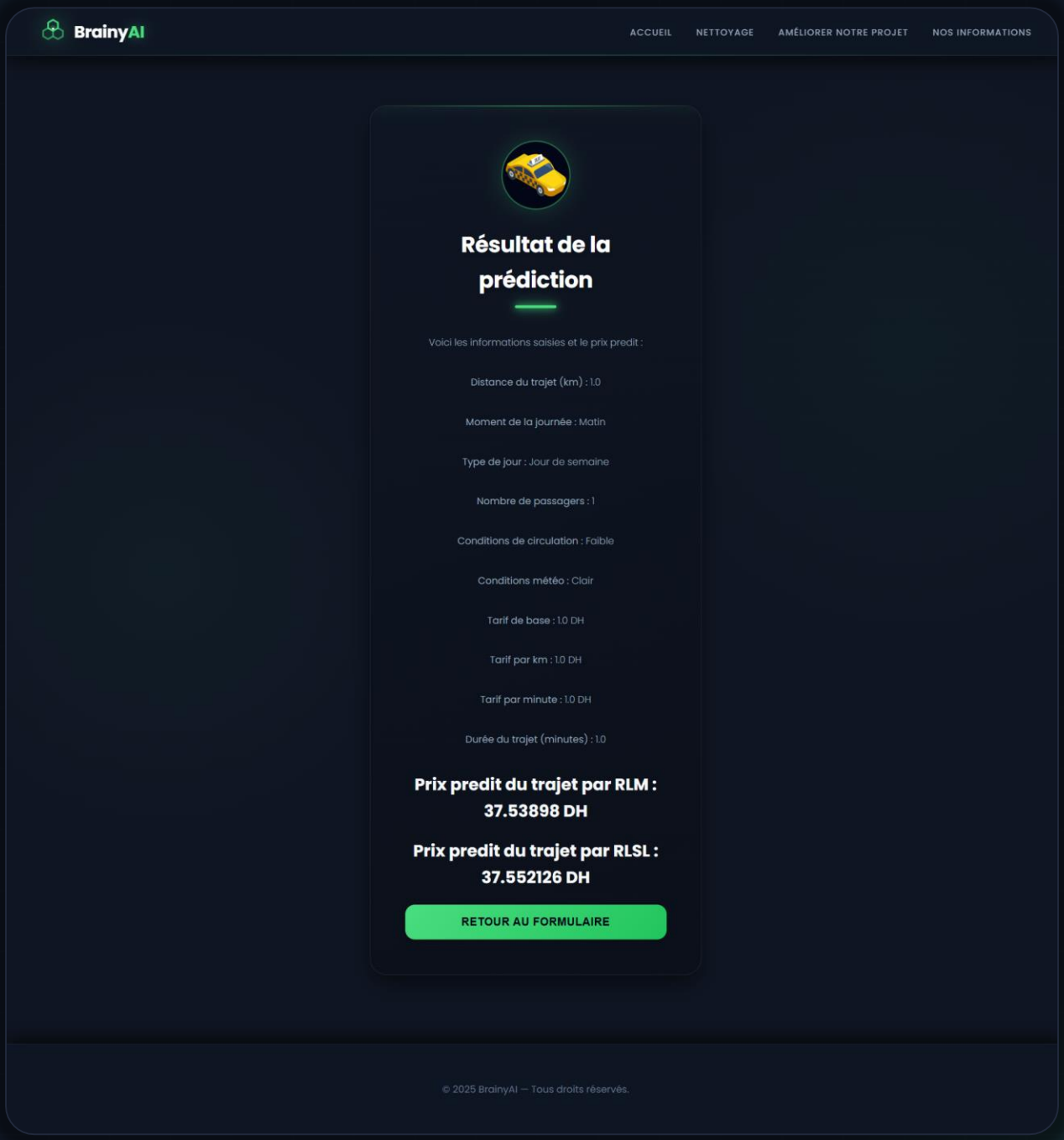
### Page de formulaire (Classification):



# Résultat :

## Captures d’écran de l’application :

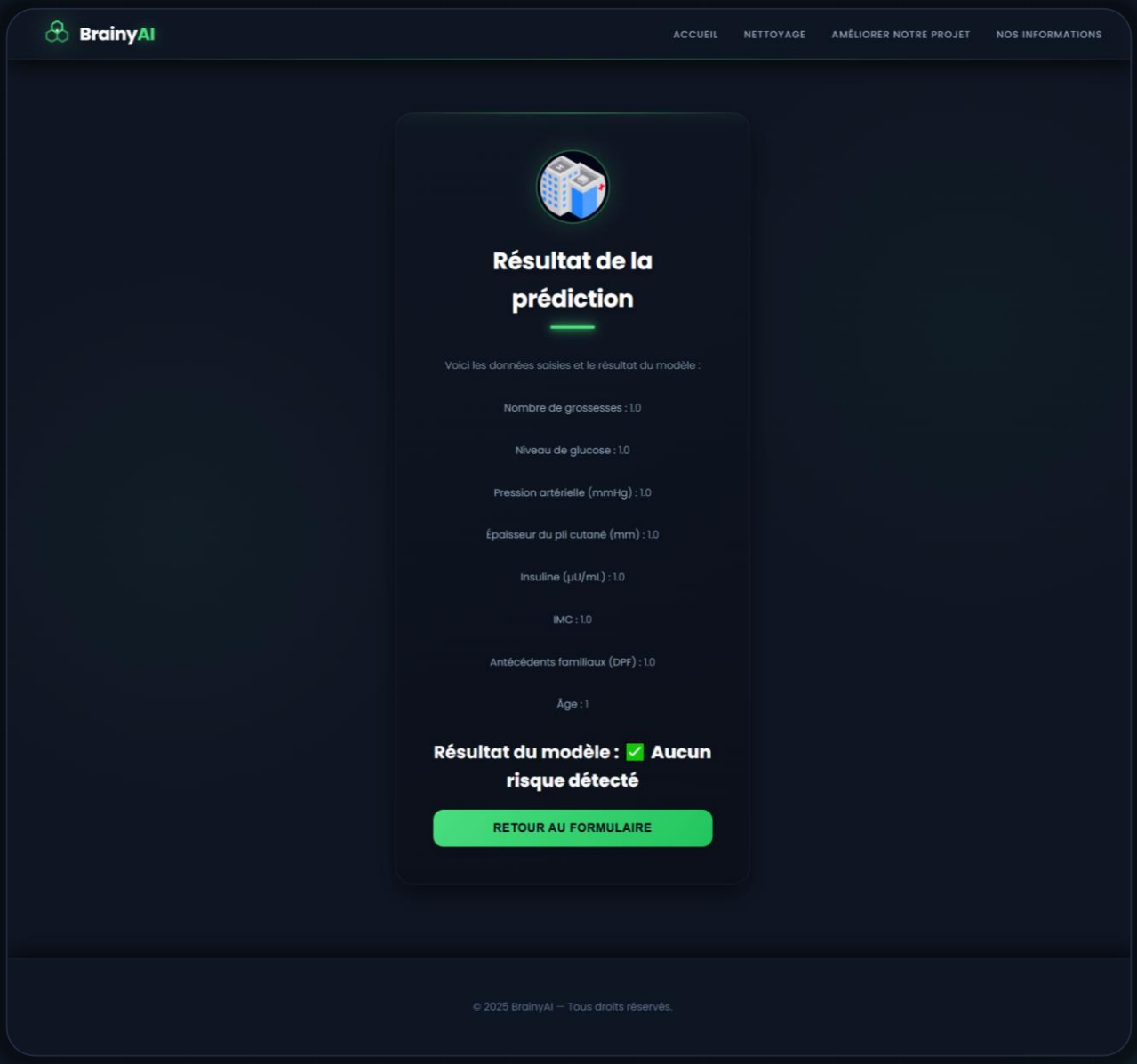
### Page de Résultat (Régression):



# Résultat :

## Captures d’écran de l’application :

### Page de Résultat (Classification):



# Résultat :

## Captures d'écran de l'application :

### Page de Nettoyage:

Le programme de nettoyage utilisé dans notre projet est relativement basique et ne couvre pas tous les cas possibles, mais il peut être facilement amélioré et étendu dans le futur. Son rôle principal est de préparer les données brutes afin qu'elles soient exploitables par les modèles de Machine Learning.

**Processus de nettoyage :**

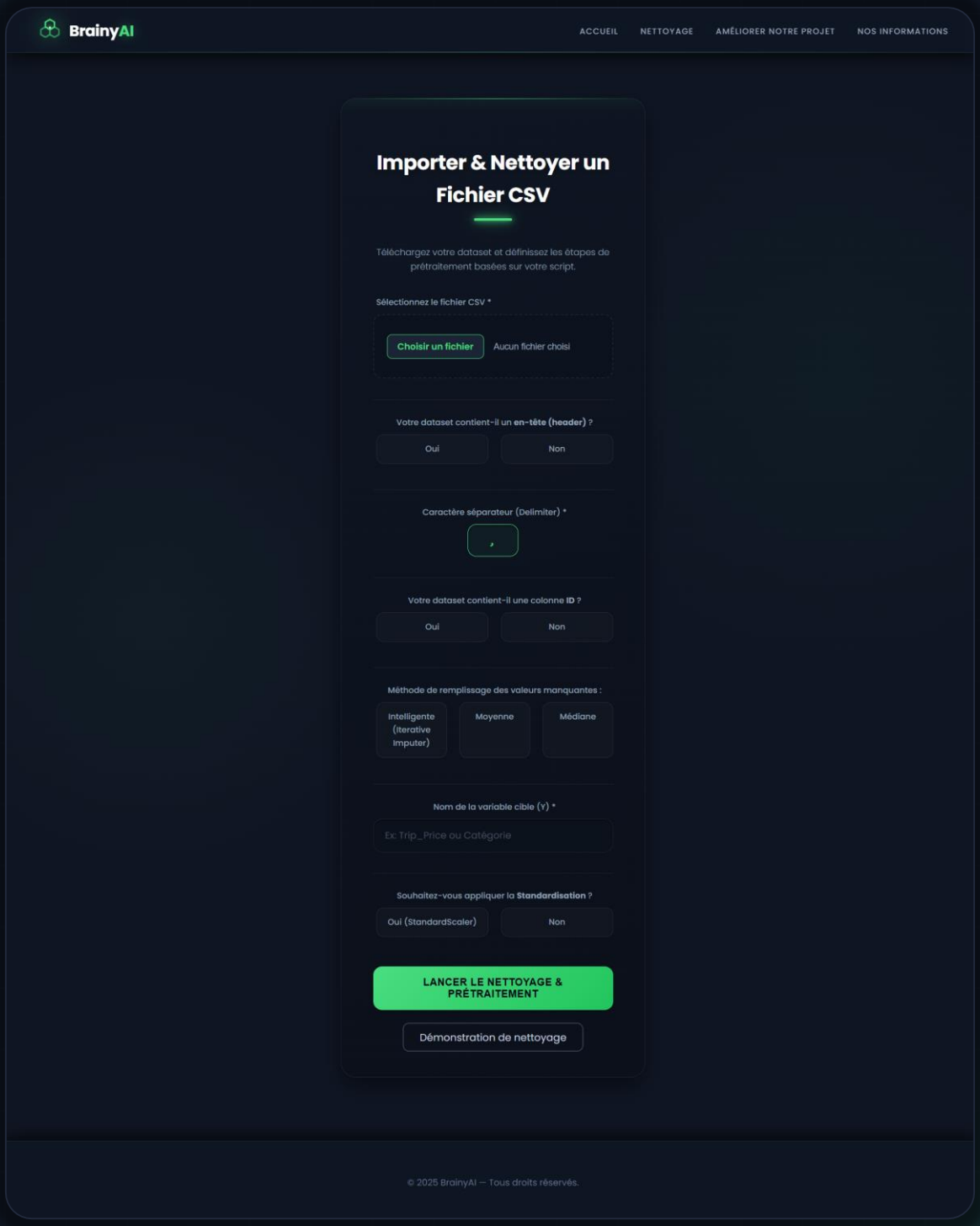
1. Importation du fichier CSV :
   - L'utilisateur sélectionne le fichier à traiter et choisit également le délimiteur utilisé.
   - Deux cas sont possibles :
     1. **Si le fichier contient un en-tête (header), il n'est pas nécessaire de le saisir.**
     2. **Sinon, l'utilisateur doit fournir les noms des colonnes manuellement.**
     3. **Séparation des colonnes catégorielles et numériques**
2. Colonnes catégorielles :
   - Les colonnes de type date sont prises en compte si elles existent.
   - Les valeurs manquantes sont remplacées par la mode (valeur la plus fréquente).
   - Les colonnes sont ensuite encodées pour être transformées en valeurs numériques, permettant leur utilisation par les modèles de Machine Learning.
3. Colonnes numériques :
   - Les valeurs manquantes peuvent être remplies selon plusieurs méthodes choisies par l'utilisateur :
     1. **Moyenne**
     2. **Médiane**
     3. **Imputation avancée (IterativeImputer)**
4. Suppression des doublons
   - Les observations dupliquées sont supprimées pour éviter les biais dans l'entraînement.
5. Traitement des valeurs aberrantes (outliers)
   - La méthode du capping est appliquée : les valeurs inférieures à la borne basse sont remplacées par cette dernière, et de même pour la borne haute.
6. Séparation des variables
   - Les colonnes indépendantes (X) sont séparées de la variable cible (y).
7. Standardisation (optionnelle)
   - Avant de retourner le dataset nettoyé, l'utilisateur peut choisir d'appliquer une standardisation aux colonnes indépendantes.
8. Retour du dataset final
   - Le dataset nettoyé est recombiné par concaténation horizontale de X et y, puis réindexé avant d'être renvoyé pour l'utilisation dans les modèles de Machine Learning.

Cette page assure un nettoyage complet et structuré des données, tout en offrant une certaine flexibilité à l'utilisateur pour choisir les méthodes les plus adaptées à son jeu de données.

# Résultat :

## Captures d’écran de l’application :

### Page de Nettoyage:



# Résultat :

## Captures d'écran de l'application :

### Page de Nettoyage:

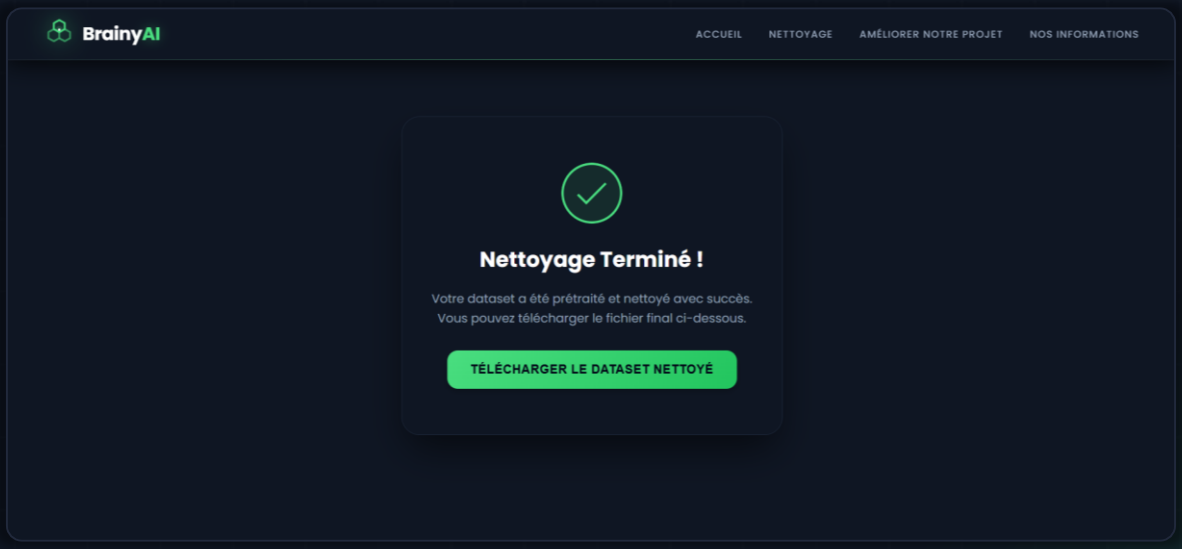
Une fois toutes les étapes de nettoyage terminées (gestion des valeurs manquantes, traitement des valeurs aberrantes, suppression des doublons, encodage des variables catégorielles et éventuelle standardisation), l'utilisateur est redirigé vers la page de résultat du cleaning.

Cette page joue un rôle très important dans le processus, car elle permet à l'utilisateur de visualiser que le nettoyage a bien été effectué avec succès. Elle agit comme une confirmation finale du traitement appliqué sur le dataset initial. Le système affiche un message indiquant que les données ont été correctement nettoyées et préparées pour une utilisation ultérieure.

Un bouton de téléchargement est mis à disposition sur cette page. Ce bouton permet de récupérer facilement le nouveau fichier contenant le dataset nettoyé. Ce fichier mis à jour est débarrassé des incohérences, des valeurs manquantes et des valeurs aberrantes non traitées, ce qui le rend beaucoup plus adapté à l'étape suivante du projet, notamment l'entraînement des modèles de Machine Learning et l'évaluation de leurs performances.

Grâce à cette fonctionnalité, l'utilisateur n'a pas besoin de nettoyer à nouveau ses données ou d'utiliser des outils externes. Il peut télécharger directement le fichier propre et l'intégrer dans ses futurs travaux d'analyse, de visualisation ou de prédiction. Cette page rend donc le processus plus fluide, plus rapide et plus accessible même pour des utilisateurs ayant peu de connaissances techniques en traitement de données.

Enfin, cette page contribue à rendre l'application plus intuitive et plus professionnelle, en offrant une expérience utilisateur complète allant de l'importation du dataset brut jusqu'à l'obtention d'un jeu de données prêt à l'emploi.



# Résultat :

## Captures d’écran de l’application :

### Page de collecte de nouvelles données réelles :

L'interaction utilisateur débute par l'apparition d'une fenêtre modale (popup) au design moderne et sombre, intitulée "Contribuer au projet". Illustrée par une icône de fusée symbolisant l'avancement du projet, cette interface invite l'utilisateur à participer activement en partageant ses informations. Elle présente deux options claires : un bouton "PLUS TARD" pour reporter l'action, et un bouton vert mis en évidence "AJOUTER MES INFOS".

En cliquant sur ce dernier, l'utilisateur accède directement à la fonctionnalité d'enrichissement de la base de données. En effet, en plus de la page de nettoyage des données, la plateforme **BrainyAI** intègre cette page dédiée à la collecte de nouvelles données réelles. Cette fonctionnalité permet d’enrichir progressivement les jeux de données existants afin d’améliorer la qualité et la précision des modèles de prédiction.

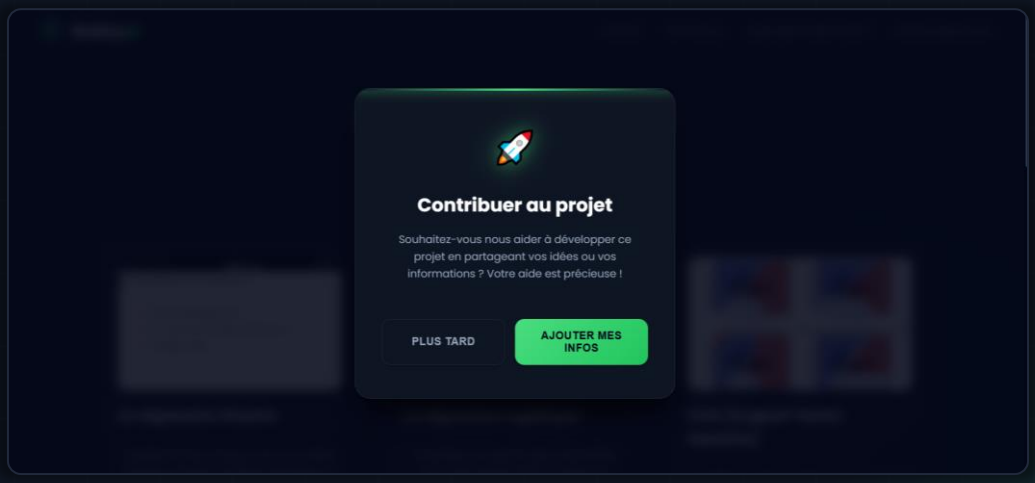
Cette page se présente sous la forme d’un formulaire interactif dans lequel l’utilisateur est invité à saisir l’ensemble des informations correspondant aux différentes colonnes du dataset, y compris la variable cible (par exemple Trip_Price pour le dataset Taxi). Chaque champ du formulaire représente une caractéristique spécifique du jeu de données, garantissant ainsi la cohérence et la structure des données collectées.

Une fois le formulaire validé, un script Python dédié se charge de :

- Enregistrer automatiquement ces nouvelles données dans un fichier nommé taxi_new_data.xlsx.

Ce fichier peut ensuite être exploité pour : **Étendre** le jeu de données original, **Réentraîner** les modèles existants, **Améliorer** la précision des prédictions à long terme.

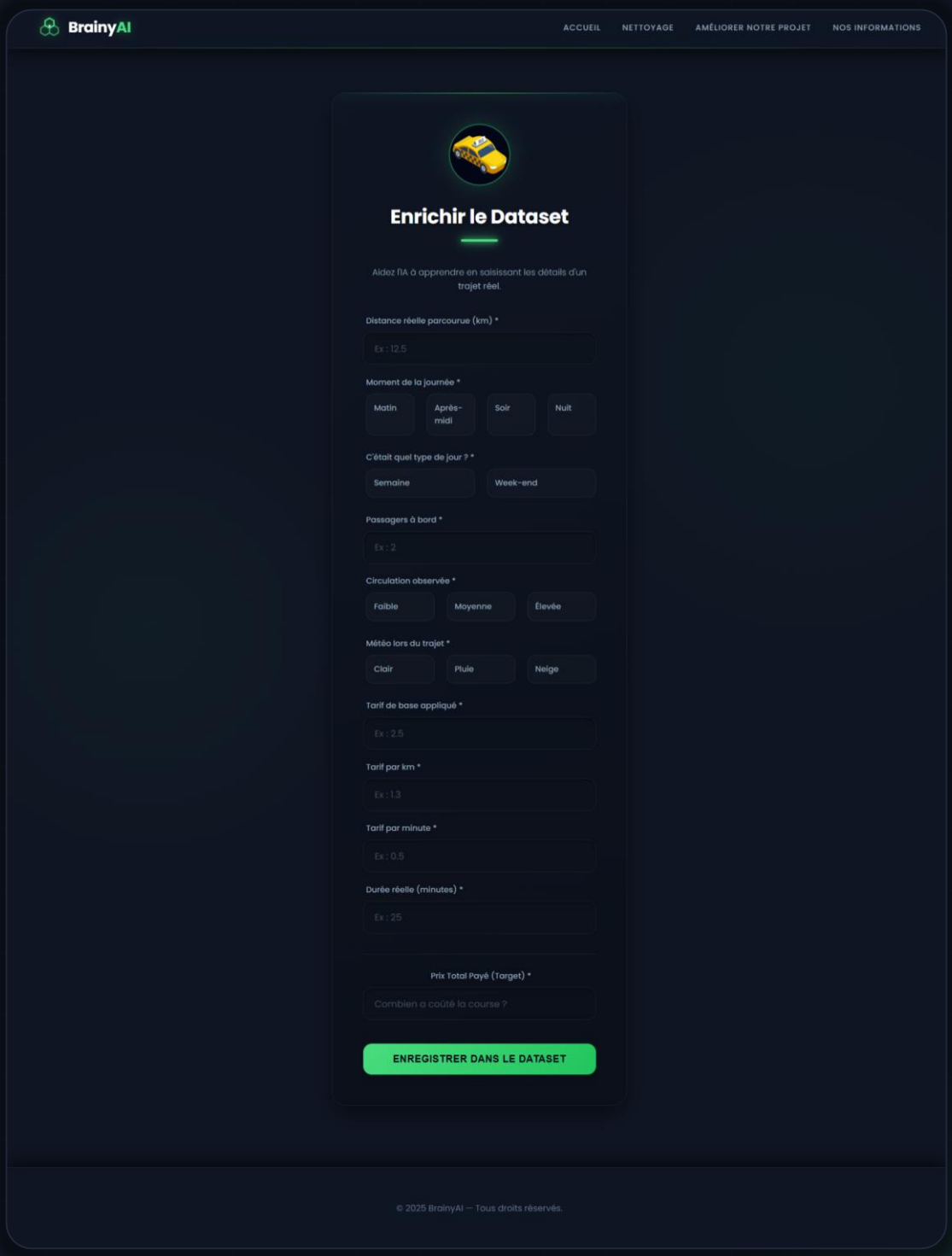
Cette fonctionnalité ajoute une dimension évolutive à la plateforme, en permettant une amélioration continue des performances des modèles grâce à l’intégration de données réelles issues des utilisateurs.



# Résultat :

## Captures d’écran de l’application :

### Page de collecte de nouvelles données réelles :



# Résultat :

## Captures d'écran de l'application :

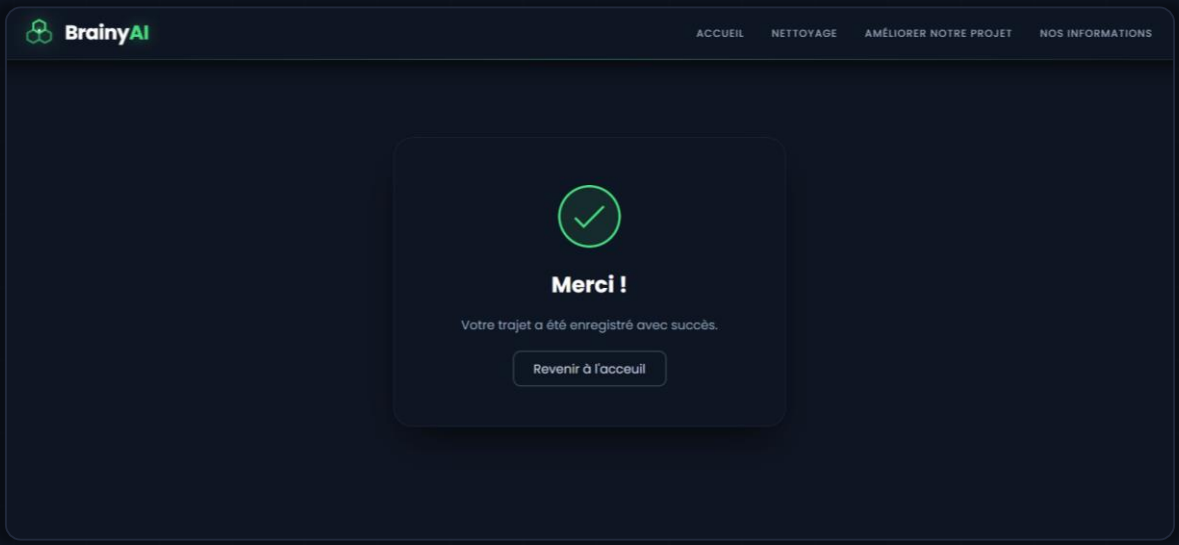
### Page de collecte de nouvelles données réelles :

Cette page est affichée après la soumission réussie du formulaire d'ajout de nouvelles données liées à un trajet de taxi. Elle sert de confirmation visuelle et fonctionnelle que les informations saisies par l'utilisateur ont bien été enregistrées dans le fichier taxi_new_data.xlsx.

Au centre de la page se trouve une carte de confirmation au design moderne, contenant :

- Un message principal "Merci !", qui renforce l'interaction positive avec l'utilisateur.
- Un texte explicatif : « Votre trajet a été enregistré avec succès. », qui confirme clairement la bonne prise en compte des données soumises.
- Sous ce message, un bouton "Revenir à l'accueil" est proposé.

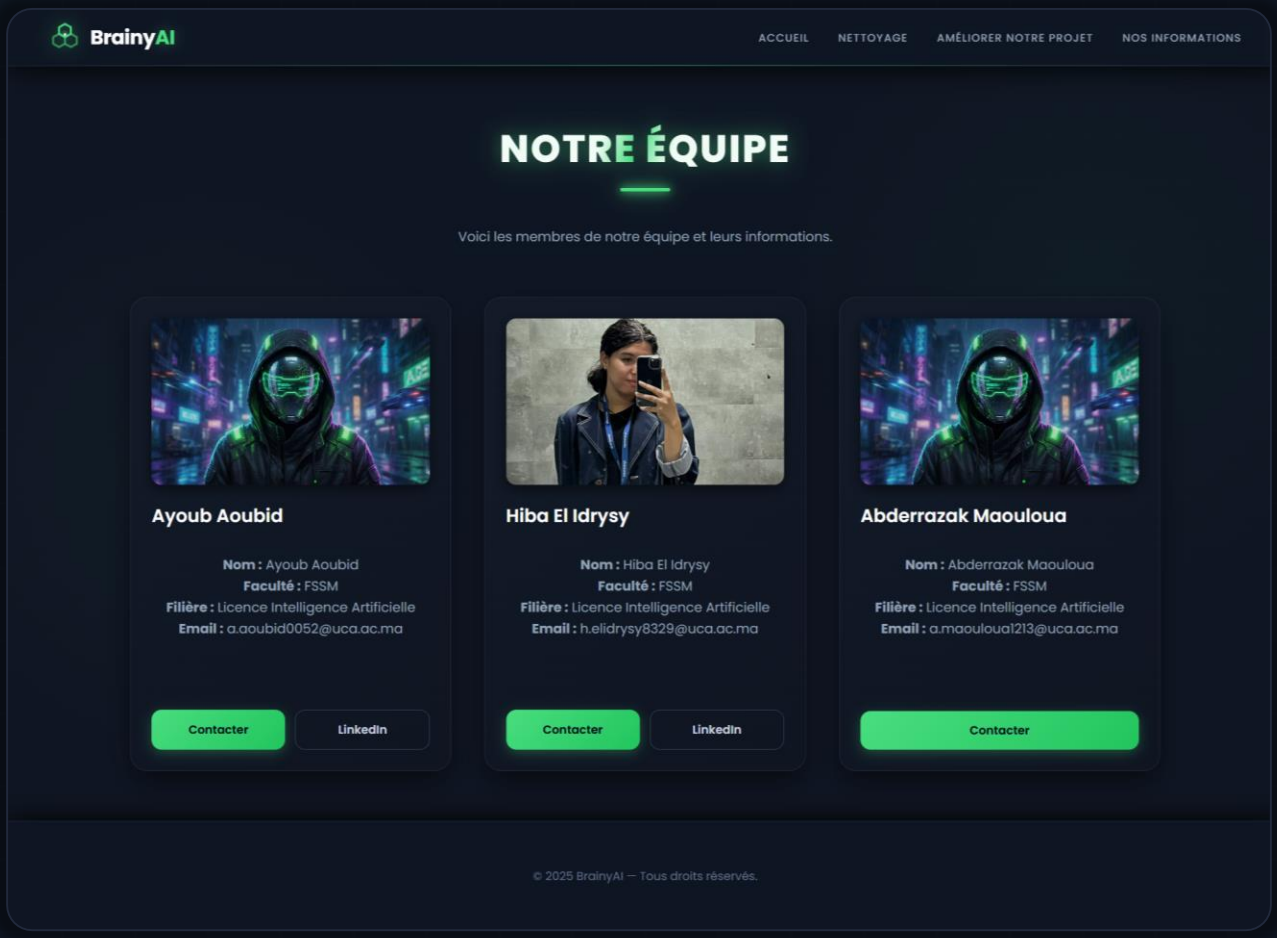
Celui-ci permet à l'utilisateur de retourner facilement à la page principale de la plateforme **BrainyAI** afin d'accéder aux autres fonctionnalités disponibles, comme le nettoyage de données, la prédiction ou l'amélioration du projet.



# Résultat :

## Captures d’écran de l’application :

### Page d'équipe :



# Conclusion et Perspectives :

Ce projet a permis de concevoir et de mettre en place une plateforme web interactive, nommée **BrainyAI**, dédiée à l'expérimentation et à l'évaluation de plusieurs algorithmes d'apprentissage automatique. À travers son développement, nous avons pu combiner des concepts théoriques du Machine Learning avec une application pratique intégrée dans un environnement web basé sur le framework **Django**. La plateforme permet non seulement de réaliser des prédictions à partir de modèles entraînés, mais également de nettoyer et préparer les données via une interface dédiée.

En complément de la page de nettoyage, une page de collecte de nouvelles données réelles a également été développée. Celle-ci permet aux utilisateurs de saisir et de stocker de nouvelles observations, qui pourront être utilisées ultérieurement pour enrichir les jeux de données existants et améliorer progressivement la qualité ainsi que la précision des prédictions.

## Bilan du projet :

Les objectifs initiaux ont été globalement atteints. Nous avons réussi à :

- Implémenter plusieurs algorithmes de Machine Learning, notamment la régression linéaire et logistique, les arbres de décision, Random Forest, SVM et XGBoost.
- Appliquer des techniques de prétraitement des données telles que la gestion des valeurs manquantes, le traitement des valeurs aberrantes, la normalisation et l'encodage.
- Intégrer ces modèles dans une plateforme web fonctionnelle, intuitive et accessible.
- Développer une page dédiée à la collecte de nouvelles données réelles, contribuant à l'enrichissement progressif des datasets et à l'amélioration des performances des modèles.
- Évaluer les performances des algorithmes à travers deux cas d'usage réels : la prédiction du prix d'un trajet de taxi (régression) et le diagnostic du diabète (classification).

Ce projet a ainsi permis de mettre en évidence l'importance de la qualité des données et du choix de l'algorithme dans l'obtention de prédictions fiables.

## Compétences acquises :

La réalisation de ce projet nous a permis de développer et de renforcer plusieurs compétences, notamment :

- La compréhension et l'implémentation des principaux algorithmes de Machine Learning.
- La maîtrise des étapes de prétraitement des données (data cleaning).
- Le développement d'une application web avec le framework **Django**.
- La gestion, la collecte et la manipulation de jeux de données réels.
- L'évaluation et la comparaison des performances de différents modèles.
- L'amélioration des capacités d'analyse, de résolution de problèmes et de travail autonome.

## Limites du projet :

- **Données restreintes :** La taille réduite des jeux de données freine la capacité de généralisation des modèles.
- **Optimisation perfectible :** Le réglage des hyperparamètres n'est pas entièrement automatisé, laissant une marge de progression pour certains modèles (XGBoost, SVM).
- **Fonctionnalités limitées :** La plateforme propose encore peu d'algorithmes et des visualisations basiques.
- **Collecte incertaine :** La qualité et la diversité des futures données dépendent fortement de la participation et de la rigueur des utilisateurs.